

M2: SQL

"Structured Query Language" es el lenguaje estándar para manipular datos en bases de datos relacionales. Es NO PROCEDIMENTAL (qué y) / DECLARATIVO (NO cómo). Permite llevar a cabo procesos ETL: (Extract, Transform and Load) integra, obtiene y manipula datos de múltiples fuentes

→ Query: ^{declaración} instrucción "consulta" para seleccionar un grp. de datos de la $\mathbb{D}$ para después manipular los mismos con cualquier acción

- SIEMPRE debe incluir (al inicio) un "select" + indica la(s) columna(s) a consultar (* si serán todas) + "from" + indica la(s) tabla(s) a consultar

- Se debe tener cuidado con no sobrecargar el sistema al querer consultar un no. excesivo de datos. Se pueden agregar límites ("limit")

- Adicional se le pueden añadir más instrucciones de filtro y acomodo para la selección.

→ select atr1, atr2, atr3 → selecciona esos atributos

→ from table 1 → selecciona de esa(s) tabla(s)

→ where atr2 = 'A' or atr2 = 'B' → buscar atr. % condición

→ group by atr3 → agrupa registros con mismo valor en atr3

↳ having → agrega una condición numérica. los valores del atributo

→ order by atr3 → x default ordena x ascendente el atributo

- Si se añade "distinct" después de select, NO se repiten los valores iguales

- Patrones: sirven para especificar una condición sobre el texto/etiqueta buscado → se buscan con "LIKE"

→ %: ignorar caracteres ej: where city like '%n' = que terminen con 'n'

→ _: ignorar 1 caracter ej: city('') '%n_' = penúltima letra "n"

→ Puede hacer uso de ¹⁰⁵ operadores OR, AND y NOT, pero también de "in": revisa si un atributo o variable es = a valor(es) específicos. ☆☆☆
ej: Letter in ('A', 'B', 'C') (es = a?) ó NOT in (...) (es ≠ a?).

- También se puede usar **NULL** ó **"IS NULL"** para buscar y seleccionar atributos sin valor (nulos) (o **"IS NOT NULL"** para atributos con valor)
- **"DESC"**: si se agrega después de **"order by"** el atributo ordena de forma descendiente (en lugar de ascendente)

8/10/25

- **"COUNT"**: regresa no. de valores no NULL que existen en esa columna/atributo.
 - Si se usa con **as** puede guardarse en un atrib. nuevo
 - Si se usa con la PK, sirve para contar cada entidad integrante 1 sola vez (no sus apariciones)
- **"COUNT(*)"**: no. de registros en la tabla seleccionada. No elimina valores NULL.

- **"MIN"**: regresa el valor mínimo en esa columna. ~~no~~ eliminan NULLs

- **"MAX"**: regresa el valor máximo de la columna

- **"SUM"**: regresa la suma de los valores en esa columna. Elimina NULLs.

- **"AVG"**: regresa el promedio de los valores en esa columna. Elimina NULLs.

- Si se añade **"distinct"** antes del atributo, solo devuelve valores distintos (elimina duplicados)
- **"count (distinct Atr)"** = conteo de integrantes 1 sola vez

- **MAX, MIN, SUM, AVG y COUNT SOLO** se usan con el **"select"** y con **"HAVING"** (condición numérica)

- **"as"**: le asigna un nombre al valor(es) encontrado(s) → resultado de la búsqueda/consulta

- **"JOIN"**: * combina registros de 1 o + tablas de una BD en una tabla. Iguala la PK y la FK de las
- * (o **"INNER JOIN"**) ☆☆☆

tablas (relación). También se puede hacer con un **WHERE**.

```
SELECT atr1, table1.atr2, atr3  
FROM table1 INNER  
JOIN table2  
ON table1.atr2 = table2.atr2
```

(Donde atr2 es la PK/FK)

FROM
WHERE table1.atr2 = table2.atr2

9/10/25

→ (Cuando se usa un atributo presente en +1 de las tablas seleccionadas, se especifica de qué tabla se selecciona con "tabla.atributo")

→ Si se añade un nombre/letra después del **FROM** y de la tabla, esos se convierten en sus alias (para usar en el query)

→ **"UNION"**: combina los resultados/registros de consulta de 2 tablas (con mismo no. de columnas de resultado). Devuelve todos los registros de ambas, pero elimina duplicados.
($r \cup s = \text{"select * from r union select * from s"}$).

→ **"EXCEPT"**: devuelve la diferencia de los registros resultado de 2 tablas (presentes en 1^{er} tabla pero no en la 2^{da}), $r - s = \text{"select * from r except select * from s"}$. Elimina duplicados.

→ **"INTERSECT"**: devuelve los registros compartidos^(por ^) presentes en ambas tablas. ($r \cap s = \text{"select * from r intersect select * from s"}$). Elimina duplicados.

→ **Producto cartesiano**: combina cada registro de la 1^{era} tabla con cada registro de la 2^{da} → devuelve todas las combinaciones posibles. ($r \times s = \text{"select * from r, s"}$ ó $\text{"select * from r CROSS JOIN s"}$). No elimina duplicados.

My day in colour:



→ "EXISTS": revisa que las tuplas/registros con la condición existan → o que no existan:

"NOT EXISTS": Evalúa 1 x 1 a TRUE/FALSE. ^{para añadirla.}

→ Crear una tabla: "CREATE TABLE tabla1 (atributos: type de los atr. not null, ...)"
SE DEBEN ESPECIFICAR. NO IMPORTA EL ORDEN, SU VALOR ES = 1

→ Agregar un atributo a una tabla: "ALTER TABLE tabla1 ADD atr1 type del atr"

→ Borrar una tabla: "DROP TABLE tabla1"

→ Agregar un nuevo registro/fila a la tabla:

"INSERT INTO table1 (atr1, atr2...) VALUES (^{atr1}1, ^{atr2}"A", ...)" ó a partir de los resultados de un query.

→ Eliminar filas/registros de la tabla: "DELETE FROM table1 WHERE..."

→ Modificar valores en registros que cumplen una condición: "UPDATE table1 SET atr1 = "nuevoValor" WHERE [condición]"

→ DDL (Data Definition Language): comandos que se usan para definir, modificar y ^{restringir} gestionar la estructura de objetos de bases de datos (tablas, índices, vistas y esquemas. → Organización física y ^{esquema} de la BD (definir columnas/atributos).

- Sus declaraciones son generalmente ejecutadas por los DBA (administradores BD), irreversibles y ejecutados con menor frecuencia

- Sus comandos NO manipulan los datos (contenido) directamente

- Ejemplos: 'CREATE', 'DROP', 'ALTER', 'RENAME', 'TRUNCATE'

→ DML (Data Manipulation Language): comandos que se usan para manipular los datos en sí (contenido) de los objetos de la BD → definir las tuplas (añadir, ^{consultar} ~~recuperar~~ o actualizar los datos)

- Sus declaraciones son generalmente ejecutadas por los desarrolladores de apps o usuarios de la BD, reversibles y ejecutados con mayor frecuencia.

- Sus comandos usan condiciones

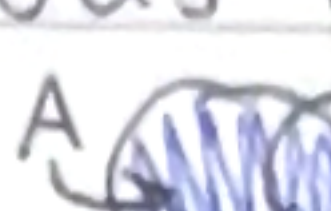
- Ejemplos: 'UPDATE', 'INSERT', 'MERGE', 'SELECT', 'DELETE'

24/11/25

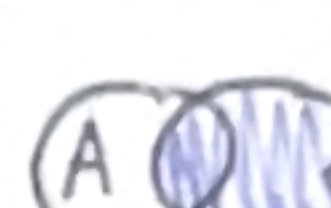
SQL JOIN & UNION

→ Joins: combinan filas de '2' ó + tablas en una base de datos a partir de un atributo =

◦ INNER JOIN: devuelve solo las filas que tienen coincidencias en ambas tablas 

◦ LEFT JOIN: devuelve todas las filas de la tabla izq. y las coincidentes  (FROM A LEFT JOIN B)

◦ RIGHT JOIN: todas

derecha y las coincidentes  (FROM A RIGHT JOIN B)

◦ FULL ^{OUTER} JOIN: devuelve todas las filas de ambas tablas (incluyendo las coincidentes) 

El atributo común se define usando "ON":

```
SELECT employees.name, departments.name
```

```
FROM employees
```

```
INNER JOIN departments (ON) employees.departmentID =  
departments.departmentID
```

→ Unions: unión de conjuntos para combinar los resultados de 2 ó + consultas select

```
SELECT ... FROM ...
```

```
UNION ←
```

```
SELECT ... FROM ...
```

Se utilizan 2 veces la tabla (aliases ≠) →

```
SELECT t1.c1, t2.c2  
FROM tabla as t1,  
tabla as t2
```

→ Autocombinación: unión de una tabla consigo misma al comparar 2 de sus col. con = tipo de datos (ej: de tabla empleados unir el supervisor (empleado) en el registro del empleado).

→ SELF JOIN: producto cartesiano (CROSS JOIN) de una tabla consigo misma (bajo condiciones)

→ "<>": Comparador-operador equivalente a '≠'.

